

Computer Organization & Assembly Language (CSC-236)

Lecture 11

4

Bit Manipulations

4.1. MULTIPLICATION ALGORITHM

With the important capability of decision making in our repertoire we move on to the discussion of an algorithm, which will help us uncover an important set of instructions in our processor used for bit manipulations.

Multiplication is a common process that we use, and we were trained to do in early schooling. Remember multiplying by a digit and then putting a cross and then multiplying with the next digit and putting two crosses and so on and summing the intermediate results in the end. Very familiar process but we never saw the process as an algorithm, and we need to see it as an algorithm to convey it to the processor.

To highlight the important thing in the algorithm we revise it on two 4bit binary numbers. The numbers are 1101 i.e. 13 and 0101 i.e. 5. The answer should be 65 or in binary 01000001. Observe that the answer is twice as long as the multiplier and the multiplicand. The multiplication is shown in the following figure.

```
1101 = 13
0101 = 5
-----
1101
0000x
1101xx
0000xxx
-----
01000001 = 65
```

4.2. SHIFTING AND ROTATIONS

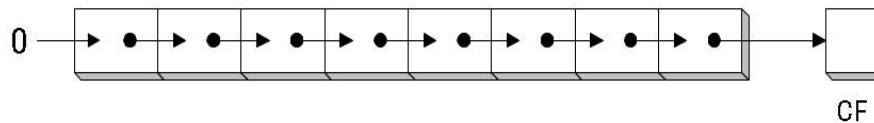
- The set of shifting and rotation instructions is one of the most useful set in any processor's instruction set
- They simplify really complex tasks to a very neat and concise algorithm
- The shifting and rotation operations which are available in our processor are mentioned in the next slide

Shift and Rotate Instructions

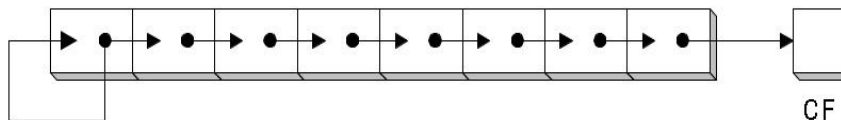
- Shift instructions are among the most characteristic of assembly language.
- Bit shifting means to move bits right and left inside an operand.

LOGICAL VS ARITHMETIC SHIFTS

- A logical shift fills the newly created bit position with zero:



- An arithmetic shift fills the newly created bit position with a copy of the number's sign bit:



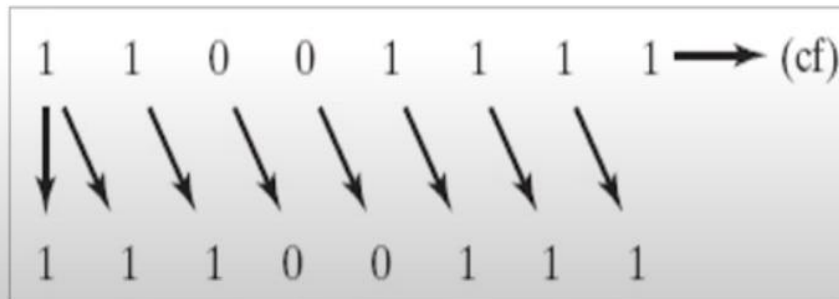
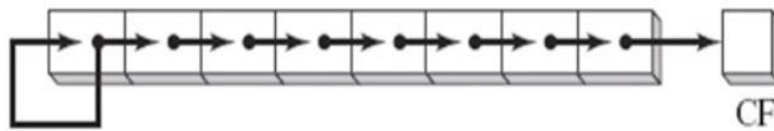
Logical Shift

- Logical shift fills the newly created bit position with zero.
- In the following illustration, a byte is logically shifted one position to the right.
- In other words, each bit is moved to the next lowest bit position. Note that bit 7 is assigned 0:



Arithmetic Shift

- The newly created bit position is filled with a copy of the original number's sign bit.



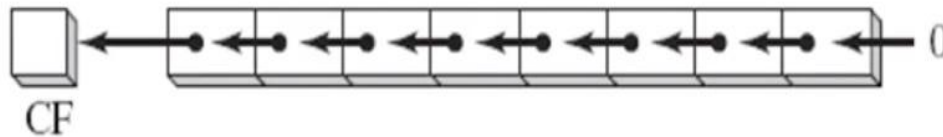
Shift and Rotate Instructions

SHL	Shift left
SHR	Shift right
SAL	Shift arithmetic left
SAR	Shift arithmetic right
ROL	Rotate left
ROR	Rotate right
RCL	Rotate carry left
RCR	Rotate carry right
SHLD	Double-precision shift left
SHRD	Double-precision shift right

1.) Shift Left/ Shift Logical Left (SHL)

SHL Instruction

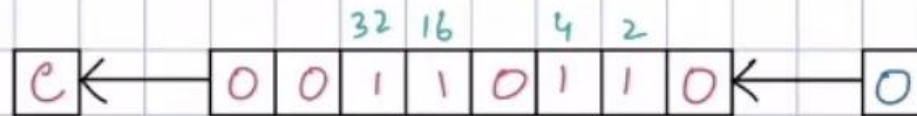
- The SHL (shift left) instruction performs a logical left shift on the destination operand, filling the lowest bit with 0.
- The highest bit is moved to the Carry flag, and the bit that was in the Carry flag is discarded.



1.) Shift Left/ Shift Logical Left (SHL)

Shift Logical Left (SHL) / Shift Arithmetic Left (SAL)

54



108



1.) Shift Left/ Shift Logical Left (SHL)

SHL Instruction

- The first operand in SHL is the destination and the second is the shift count.
- x86 processors permit imm8 to be any integer between 0 and 255. Alternatively, the CL register can contain a shift count.

`SHL destination, count`

`SHL reg, imm8`

`SHL mem, imm8`

`SHL reg, CL`

`SHL mem, CL`

1.) Shift Left/ Shift Logical Left (SHL)

SHL Instruction: Example

- In the following instructions, BL is shifted once to the left.
- The highest bit is copied into the Carry flag and the lowest bit position is assigned zero.

```
mov  bl,8Fh                ; BL = 10001111b
shl  bl,1                   ; CF = 1, BL = 00011110b
```

- When a value is shifted leftward multiple times, the Carry flag contains the last bit to be shifted out of the most significant bit (MSB).

```
mov  al,10000000b
shl  al,2                   ; CF = 0, AL = 00000000b
```

1.) Shift Left/ Shift Logical Left (SHL)

SHL Instruction: Bitwise Multiplication

- Bitwise multiplication is performed when you shift a number's bits in a leftward direction (toward the MSB). For example, SHL can perform multiplication by powers of 2.
- Shifting any operand left by n bits multiplies the operand by 2^n .

```
mov  dl,5      Before: 0 0 0 0 0 1 0 1 = 5
shl  dl,1      After:  0 0 0 0 1 0 1 0 = 10
```

```
mov  dl,10      ; before: 00001010
shl  dl,2      ; after:  00101000
```


2.) Shift Right / Shift Logical Right (SHR)

SHR Instruction

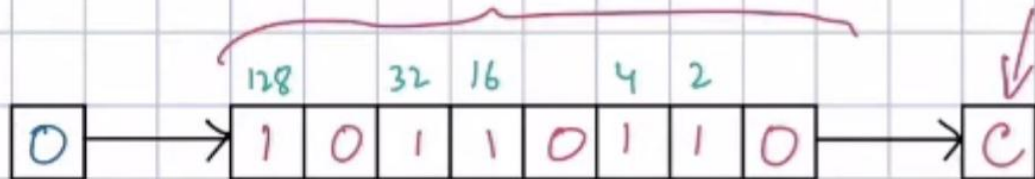
- The SHR (shift right) instruction performs a logical right shift on the destination operand, replacing the highest bit with a 0.
- The lowest bit is copied into the Carry flag, and the bit that was previously in the Carry flag is lost.
- SHR uses the same instruction formats as SHL.



2.) Shift Right / Shift Logical Right (SHR)

Shift Logical Right (SHR)

1 8 2



9 1



2.) Shift Right / Shift Logical Right (SHR)

SHR Instruction: Example

```
mov al,0D0h           ; AL = 11010000b
shr al,1               ; AL = 01101000b, CF = 0

mov al,00000010b
shr al,2               ; AL = 00000000b, CF = 1
```

2.) Shift Right / Shift Logical Right (SHR)

SHR Instruction: Bitwise Division

- Bitwise division is accomplished when you shift a number's bits in a rightward direction (toward the LSB).
- Shifting an unsigned integer right by n bits divides the operand by 2^n .
- Division of signed numbers by shifting is accomplished using the SAR instruction because it preserves the number's sign bit.

`mov dl,32` Before:

0	0	1	0	0	0	0	0
---	---	---	---	---	---	---	---

 = 32

`shr dl,1` After:

0	0	0	1	0	0	0	0
---	---	---	---	---	---	---	---

 = 16

```
mov al,01000000b      ; AL = 64
shr al,3               ; divide by 8, AL = 00001000b
```

Announcement :

Quiz 3 in next week , second class:

Date: (30-4-2025) / Wednesday

Syllabus : Chap 4 (Shifting and Rotation)

End of Lecture